Computers & Graphics 128 (2025) 104202

Contents lists available at ScienceDirect

Computers & Graphics

journal homepage: www.elsevier.com/locate/cag

Technical Section

Fast simulation of soft-body deformation using connected rigid objects☆

Moonjun Chung [a], Taesoo Kwon [a,*], Yejin Kim [b,*]

[a] Department of Computer Science, Hanyang Univerity, 222 Wangsimni-ro, Seongdong-gu, Seoul, 04763, Republic of Korea
[b] School of Games, Hongik University, 2639 Sejong-ro, Jochiwon-eup, B406-2, Sejong, 30016, Republic of Korea

ARTICLE INFO

Keywords:
Soft-body deformation
Mesh rigging
Skeleton extraction
Convex decomposition
Skinned mesh

ABSTRACT

In the field of computer graphics, physics-based simulations have been actively researched for decades to represent visually realistic motions of soft-body objects. To deform stiff objects, methods such as the finite element method (FEM) and position-based dynamics (PBD) have been traditionally used for physical simulations. However, there are many situations in which it is difficult to perform interactive simulations at high speeds on relatively low-performance devices. In this paper, we propose an approach by which to undertake rapid simulations of soft-body deformation of a 3D mesh model. Assuming an input object with high damping coefficients, we approximated the simulation process using connected rigid objects. To do this, we extracted a skeletal structure from an input mesh and generated collision meshes by clustering and decomposing the skeletal voxels into convex groups. Distributing each contact force to the rigid objects properly, our approach shows that various types of object models can deform convincingly as if they were soft-body objects. Our approach is fully automated for general users and able to simulate both rigid and soft-body objects in the current animation pipeline.

1. Introduction

In the field of computer graphics, physics-based simulations have long been actively researched to represent visually realistic motions of soft-body objects. In recent years, applications of virtual reality (VR) and augmented reality (AR) have received significant attention. In such applications, it is important to provide users with an interactive and immersive experience using rapid simulations [1]. For example, in many situations, simulations must be performed on relatively low-performance devices such as head-mounted displays and smart glasses. The ability to process simulations at high speeds and to visualize realistic movements of soft-body objects, is becoming important in the recent applications.

Soft-body simulations have been actively studied by methods such as the finite element method (FEM) and position-based dynamics (PBD). The FEMs focuses on how realistic and accurate the simulation behaves. In these cases, physical characteristics are modeled with a concise formula. For example, using the implicit integral method, which is a linear formulation of a dynamic equation, has the advantage of being able to simulate models ranging from soft-body to rigid objects. In order to undertake a simulation based on this type of volumetric mesh, this method conducts collision checks between triangles and calculates the penetration depth and normal attribute for each collision point. In other words, while it estimates collisions between objects accurately, the calculation time is strongly affected by the complexity of the object shape and the number of objects to be simulated. To facilitate the calculation time, collision check operations can be performed on many objects in a large time step, which can lead to inaccurate calculations and can make it difficult to perform sophisticated simulations. Therefore, despite the many advantages of these methods, performance can be limited. On the other hand, methods using PBD are characterized by rapid physical simulations and are suitable for simulating deformable objects with low-damping physical properties [2,3]. Recently, PBD methods have shown high-quality simulation results using objects with various physical properties. However, achieving this requires a high-performance GPU and numerous time steps for a high-speed simulation.

In this paper, we propose an approach by which to undertake rapid simulations of soft-body deformation of a 3D mesh model. Assuming an input model with high damping coefficients, we defined a skeleton-structured graph from the model and generated collision meshes to serve as connected rigid objects. Given the external forces applied at a moment of collision between rigid objects, we used forward dynamics to update the changes in the collision situation. Although our simulation approximated the movement of a soft-body object, it utilized a collision system based on the linear complementarity problem

☆ This article was recommended for publication by Prof Charlie C.L. Wang.

* Corresponding authors.
E-mail addresses: chung1246@hanyang.ac.kr (M. Chung), taesoo@hanyang.ac.kr (T. Kwon), yejkim@hongik.ac.kr (Y. Kim).

https://doi.org/10.1016/j.cag.2025.104202
Received 23 August 2024; Received in revised form 5 March 2025; Accepted 8 March 2025
Available online 17 March 2025
0097-8493/© 2025 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY-NC license (http://creativecommons.org/licenses/by-nc/4.0/).

(LCP), which adopts a large time step compared to the penalty-based method used in general soft-body simulations. Therefore, a soft-body object with high stiffness can be deformed quickly by applying a stable proportional derivative (PD) servo to the joint part of the skeleton. Distributing each contact force to the rigid objects properly, our approach shows convincing deformations of various soft-body objects and achieves real-time performance as demonstrated in the experimental results.

The contributions of our approach are as follows. First, we realized rapid performance of physical simulations of rigid objects to deformable objects. To do this, we introduced a novel method that installs a skeleton inside the input mesh, classifies the mesh into multiple rigid objects, and adapts the rigid objects to the input object in the deformation simulation. This simplifies the collision checks between the deformable objects and improves the overall performance. Next, we introduce an automated rigging process that eliminates the laborious efforts and artistic skills required in a conventional animation production. A general user can easily adapt our approach to the current animation pipeline and generate animations for both rigid and soft-body objects using a single mesh model.

We have organized the paper as follows: In Section 2, we review previous studies of physics-based simulations of object motion. Section 3 summarizes the proposed approach. Sections 4 and 5 describes the rigging technique and the simulation method, respectively. We demonstrate the experimental results in Section 6 and conclude the paper with a discussion of the limitations of the proposed method in Section 7.

2. Related works

2.1. Physical simulation of rigid objects

Physics-based simulations to animate a human-like model have been studied actively for decades. Many researchers focused on improving the stability and realism of biped character locomotion [4–8]. Recently, the use of machine learning techniques such as reinforcement and manifold learning has been utilized to control diverse behaviors of physically simulated characters [9–12].

These approaches represent a character as an articulated rigid body, similar to our approach of employing interconnected rigid objects. To create an articulated character model, the surface mesh should be partitioned into multiple rigid parts and adjusted into a desired shape, which is laborious and time-consuming for general users. On the other hand, our approach provides an automated solution for the physical simulation of both rigid and soft-bodied characters.

2.2. Physical simulation of soft-body objects

Like the physical simulation of rigid objects, there has been active research on physical simulations of soft-body objects. As part of the effort to simulate soft-body movements of non-human objects realistically, there have been attempts to apply muscle models [13,14] and a dynamic controller [15] to animals. Later, such simulations were extended to thin objects such as paper, hair, and fibers [16, 17]. Müller and Chentanez presented a method that rapidly simulated volumetric elastic objects using oriented particles to approximate a mesh surface [18]. Romero et al. proposed a learning-based contact correction technique to simulate dynamic deformation in contact situations [19]. Lan et al. proposed a simulation framework that relied on medial axis transform for the rapid processing of collisions of soft-body objects [20]. Recently, projective dynamics methods were applied to real-time applications that also maintained the simulation stability of soft-body objects [21,22].

Other studies have expedited the simulation speed of the soft-body deformation process using reduced degrees of freedom (DOFs) of the character in response to internal body forces [23,24], Similarly, a reduced deformable model in a rig-space simulation was introduced to move the character's rig kinematically based on the user's intentions while secondary effects were constrained by the subspace represented by the rig [25–27].

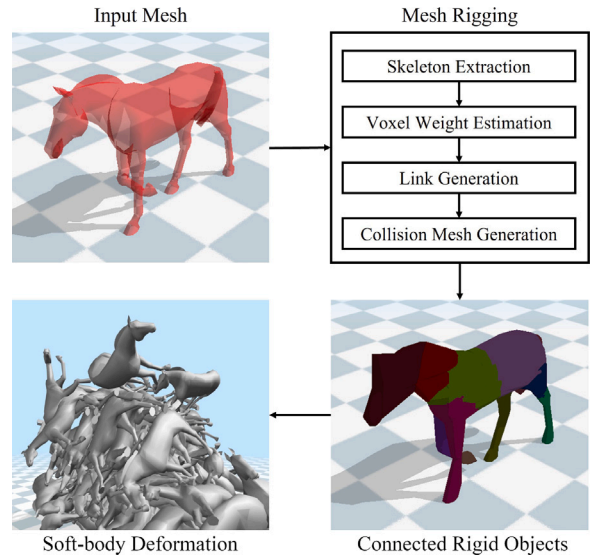


Fig. 1. Overview of the proposed simulation method for soft-body deformation using connected rigid objects.

Unlike these approaches, we use an approximation method derived from the conventional skinning technique and rigid object simulations for rapid simulations. This technique has the advantage of allowing us to apply existing skinned models directly to the current animation pipeline.

2.3. Mesh rigging

Animating a 3D mesh of a rigid object requires one to define an internal skeletal structure and attach (a.k.a. rig) it to the mesh model using skinning weights. By applying low-resolution cage meshes to estimate the skinning weights, mean-value [28], harmonic [29], and green coordinates [30] have been proposed for various application problems. However, these approaches require the cage meshes to be designed manually and require a volumetric mesh simulation akin to the FEM, which is not feasible when the desired outcome is real-time performance.

In their automatic rigging method, Baran and Popović applied a predefined skeleton to a 3D character model [31]. Au et al. introduced a simple skeleton extraction method based on mesh contraction [32]. Kavan et al. proposed an optimization method for the automatic calculation of the skinning weights [33]. Ma et al. proposed a framework of real-time skeleton extraction from sketch models [34]. In other studies, a neural network was recently applied to estimate the skin weights on a predefined skeletal structure [35–37]. However, these network-based approaches do not guarantee the rigging quality for untrained types of input models.

Our approach differs from these in that we do not require the character's rig to be prepared in advance. To the best of our knowledge, studies of physics-based simulations that automatically rig soft-body objects are scarce in the literature. Our approach can be combined with prior studies to generate more sophisticated deformation results.

3. Overview

Fig. 1 shows an overview of our approach. Given an input 3D mesh model, our approach initially undertakes an automatic rigging

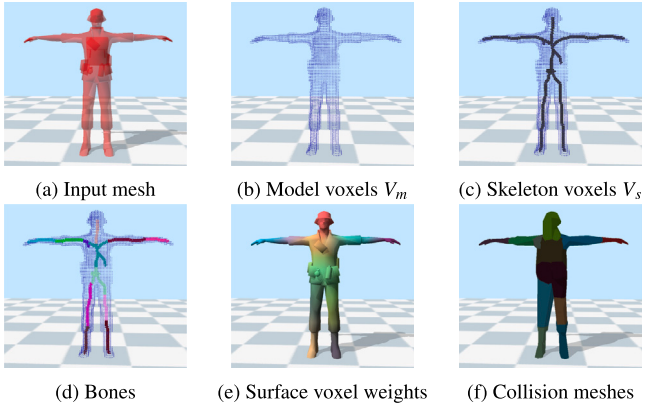


Fig. 2. Rigging Process: A mesh model is first converted into surface and interior voxels (b). Next, a graph-structured skeleton is extracted from the voxels (c). The skeleton voxels are then grouped into bones (d), which are used to derive weights (e) and collision meshes (f).

process, which consists of four major steps: skeleton extraction (Fig. 2c), voxel weight estimation (Fig. 2e), link generation (Fig. 3), and collision mesh generation (Fig. 2f). Next, the rigged object is divided into a set of connected rigid objects and is simulated physically using a PD servo, which estimates and controls the proper torques for the joints in the extracted skeleton based on the external forces applied. Details of the rigging process and physics-based simulation are described in the subsequent sections.

4. Mesh rigging

Our approach approximates a physics-based simulation of soft-body deformation using connected rigid objects, which are represented by collision meshes. This section describes the collision mesh generation process through our rigging process, which extracts a graph-structured skeleton from an input mesh, estimates the skinning weights of the voxels, and generates a collision mesh for each cluster of voxels as shown in Fig. 2.

4.1. Skeleton extraction

A graph-structured skeleton is extracted from an input mesh to construct a collision mesh. This process is divided into the following three steps. First, we generate a set of voxels of the entire model, V_m, from the input mesh, and apply a thinning algorithm on V_m to generate a set of skeleton voxels, $V_s \in V_m$. Several thinning algorithms can be applied to 3D mesh models. In our approach, we use the method of Lee et al. [38], which ensures the invariance of the Euler characteristic by checking the connectivity of each point to other points in a surrounding $3 \times 3 \times 3$ grid. Next, a graph-structured skeleton, G, consisting of a set of nodes, N_s, and edges, E_s, is created from V_m and V_s as follows. A node, $n \in N_s$, is a voxel in V_m or the source node, n_0, from which edges connect to all skeleton voxels. Specifically, edges of length 0 are connected from the source node n_0 to all nodes in V_s, ensuring that the distance from the source to every skeleton voxel is 0. Additionally, an edge, $e(n_i, n_j), i \geq 1, j \geq 1$, is created when the ith voxel lies in the surrounding grid around the jth voxel, where $e(n_i, n_j) \in E_s$. That is, $e(n_i, n_j)$ is defined if and only if the difference between the width, height, and depth values of n_i and n_j are all 0 s or 1 s, where $i \neq j$. The weight value of $e(n_i, n_j)$ is set to the value of the Euclidian distance between the two nodes (i.e., $1, \sqrt{2}, \sqrt{3}, \ldots$), to calculate the geodesic distance.

As shown in Fig. 3b, the rules described above generate G from V_s, but they do not guarantee the proper shape of V_s from diverse mesh models. For example, a thick and empty box-shaped object results in a short bar-shaped V_s, which does not represent the soft-body deformation of the box convincingly. For this reason, we used iterative steps to generate a new path in G, as follows. We find the farthest voxel, v_e, in V_m from V_s by applying the Dijkstra algorithm with n_0 as the starting point. The voxel v_e refers to the voxel farthest from the current set of skeleton voxels V_s. It is determined using the geodesic distance, which measures the distance through interior voxels only. The distance is calculated as follows:

$$D_{se}(G) = \ell(P(n_0, v_e)), \tag{1}$$

where $P(\cdot)$ is the shortest path from n_0 through all vertices of V_s to reach v_e, and $\ell(\cdot)$ is the total length of this path. The selected voxel v_e, along with all voxels along the shortest path from it to the skeleton voxels V_s, is added to V_s. G is updated for the next iteration whenever a new voxel, v_e, is added from V_m. This process is repeated until D_{se} is below a user-specified threshold value, θ_D (see Algorithm 1 for the corresponding pseudocode).

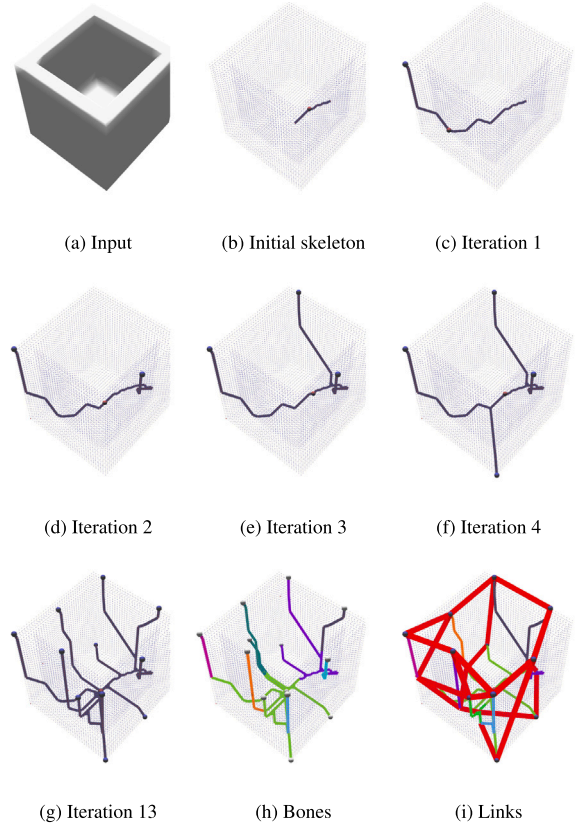
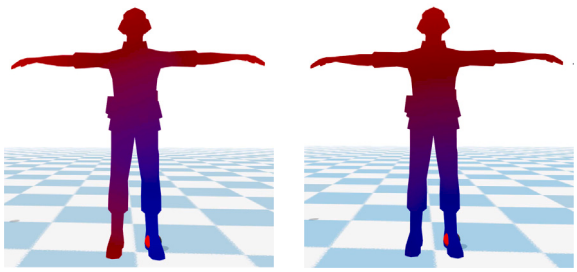


Fig. 3. Extracting a graph-structured skeleton from a box-shaped input mesh using additional iterative steps. The tree-structured skeleton is used with links, represented by red lines, to maintain its original shape.

Algorithm 1: A pseudocode for the iterative skeleton extraction process.

```
G = build (V_m, V_s) ;                          // initialization
repeat
    v_e, D_se = findFarthestNode(G, n_0);
    V_s = V_s ∪ findAllNodesInShortestPath(n_0, v_e);
    G = build(V_m, V_s) ;                       // graph update
until D_se < θ_D ;                              // threshold testing
;
```

The depth-first search (DFS) algorithm generates a minimum spanning tree of V_s and then organizes the skeleton into a spanning tree $\bar{G}$. The root node of $\bar{G}$ is set to the position of the vertex with the



(a) Geodesic distance (b) L2-distance

Fig. 4. Distance metric comparison: distances from the red sphere near the left ankle to surface voxels are visualized, with blue indicating shorter distances and red representing longer distances. Using the L2 distance metric causes both feet to be influenced by the same bone, whether in the left or right foot.

smallest sum of distances after calculating the all-pairs shortest path using the Dijkstra algorithm within the skeleton. Finally, we cluster the skeleton so that it can be used as a basis for generating convex meshes. Because we computed the all-pairs shortest path of the skeleton in the previous step, we utilized it to perform hierarchical clustering. We used the average linkage as the clustering metric and added joints to the boundaries of the clusters by moving along the minimum spanning tree of the skeleton from the root node (Fig. 3h). The reason for adding joints at the boundaries is intuitive, considering a simple case, such as rigging a long rod divided into N bones. In such a case, $N-1$ joints are installed on the boundary of each bone. The constructed bones serve as the basic units of the rigid body simulation, with each bone's mass and inertia calculated based on the voxels closest to it.

4.2. Voxel weight estimation

In the next step, the connection between surface voxels and bones is defined in a manner similar to weight painting used in animation pipelines. The simplest weight assignment method, known as rigid rigging, is explained first, followed by the method that is actually used. Rigid rigging is simply the process of identifying the set of surface voxels, V_k, where $k \in \{1, \dots, N\}$, that are closest to the kth bone. Again, a classification process without considering the geodesic distance can lead to an undesirable result. For example, for a human-shaped model, voxels of the left foot can be assigned to a bone in the right foot if using only the coordinates of the voxels (Fig. 4). To ensure an accurate classification, we use the geodesic distance between a surface voxel, $v_s \in V_m$ and a bone $B_k \subset V_s$:

$$C(s) = \arg\min_k \ell(P(n_0, B_k, v_s)), \tag{2}$$

where $P(\cdot)$ is the shortest path from the source node n_0 through bone voxels B_k to reach v_s using the Dijkstra algorithm, and ℓ is the total length of this path. Eq. (2) finds the closest bone $C(s)$ for each surface voxel v_s. However, the convex mesh should be naturally interpolated near the boundary of each bone. To do this, we applied KNN-IDW interpolation [39] to the K closest bones. To give a higher weight to a closer bone, the weight for each bone i at a surface voxel, w_{B_i}, is defined as follows:

$$\begin{cases} w_{B_i} = 0, & \text{if } D(B_i) > D(B_{K+1}), \\ w_{B_i} = \frac{1}{\max(D(B_i)^P, \epsilon)} - \frac{1}{\max(D(B_{K+1})^P, \epsilon)}, & \text{otherwise.} \end{cases} \tag{3}$$

Here, $D(B_i)$ is the distance from the surface voxel to the voxels of ith bone B_i, B_{K+1} is the $(K+1)$th nearest bone from the surface voxel, ϵ is a small positive number, and P is a parameter value used to control the degree of interpolation. It should be noted that a larger value of P results in a larger weight value for neighboring clusters, which makes the boundary of the transition from 0 to 1 sharper (i.e., the interpolation occurs more abruptly). For interpolation control, w_{B_i} is normalized using the equation below,

$$\bar{w}_{B_i} = \frac{w_{B_i}}{\sum_{i=1}^{N} w_{B_i}} \tag{4}$$

4.3. Link generation

Now that the skeleton and joints are set up, and the skinning weights for the surface voxels are defined, it is possible to animate the surface voxels or surface mesh naturally and kinematically based on given joint configurations. However, due to the tree structure of the skeleton, achieving natural deformation becomes challenging when approximating a soft-body object using a physics-based simulation of the skeleton alone. This is noticeable for objects with low stiffness coefficients. Therefore, additional links are introduced to the structure iteratively to preserve the original mesh shape. Fig. 3i shows the added links for the input mesh model.

We add links that provide connections between the node locations. Let N_j be the set of skeleton vertices in G, including all joint and leaf nodes. For every possible combination of two vertices in N_j, the distance between the nodes can be computed in two ways. The first way involves the use of the geodesic distance with an internal voxel graph, which is the shortest distance computed along the interior of the mesh. The second is to use the geodesic distance with a skeleton voxel graph, which is the shortest distance along the skeleton. If the distance calculated along the interior is closer than a user-specified threshold value (i.e., an empirical value of 1.5 in our simulation) of the distance calculated along the skeleton, it is selected as a candidate for a connecting link. To prevent redundant links from being added, we consider only the link with the shortest distance among the candidates. Once a new link is added, G is updated to include the internal geodesic path of the link, preventing the same link from being added. This process is repeated until there are no more links to add.

4.4. Collision mesh generation

In general, it is time-consuming to run a clustering algorithm on numerous voxels in an input mesh and then to group them to each collision mesh to which they belong. To accelerate this, first we group the surface voxels for which the weight differences were under the user-specified threshold value, after which we cluster them based on the weight values. Here, a common algorithm such as k-nearest neighbor can be used for clustering. To reduce the likelihood of local minima occurring during clustering, we initialize the centers of clusters by distributing them evenly across each bone. Once the cluster index was determined for each surface voxel, all the internal voxels closest to each surface-voxel cluster are determined using the Dijkstra algorithm. The cluster index of the surface voxel is copied for those internal voxels. For each set of clustered voxels, hierarchical approximate convex decomposition (HACD) was applied to generate a collision mesh [40]. Fig. 2 shows the N_c collision meshes generated from the skeleton extracted from the input mesh.

5. Physics-based simulation

In a physical approximation, a PD servo provides stable performance to represent the elasticity of objects in a simulation [41]. Using a PD controller, p-gain and d-gain can be set high, which simulates physical characteristics close to those of a rigid object using a large time step. In other words, it has advantages similar to those of the implicit integration scheme used to represent a stiff object in a FEM-based simulation. Moreover, there is no increase in the asymptotic time complexity compared to an explicit integration scheme, making it suitable for rapid simulations.

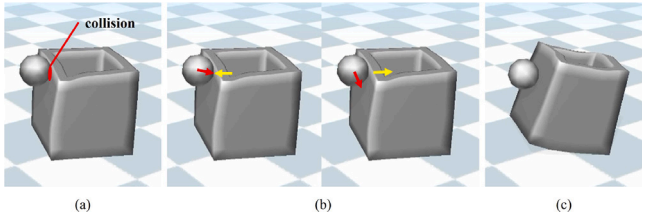


Fig. 5. Steps of the collision simulation: (a) collision detection, (b) state change, and (c) state update.

Table 1
Input triangular meshes and their resulting skeletons.

Model	Number of		
	Vertices	Triangles	Bones
Human	1198	1460	12
Box	834	1164	12
Horse	1009	1501	12

As shown in Fig. 5, the collision simulation of our approach consists of three stages. These are the collision detection, state change, and state update stages. First, the collision detection phase detects interpenetration with body parts or the ground as occurring during the simulation. As described in the previous section, we decomposed the skinned mesh into several convex meshes, running quick calculations for the collision checks between objects and proximity checks with the surrounding environment. In our approach, we applied a software library, libccd, for collision detection between two convex shapes [42].

Next, the simulation input parameters, specifically the joint torques, are computed using a PD servo. At each frame, the object's state – comprising joint configurations and their time derivatives – is updated during the simulation. Additionally, the depth of collisions detected in each collision mesh and the corresponding collision normal vectors are also updated. However, directly applying the collision impulse calculated for each contact point to a skeletal structure is challenging in our approach, as the collision mesh is transformed using linear blend skinning influenced by multiple bones. Therefore, the collision impulse is distributed to multiple bones according to the weight value ratio of the bone. The contact Jacobian used to calculate the impulse is applied to account for the influence of multiple bones:

$$\mathbf{J}^c = \sum_{i \in \mathcal{B}_c} w_{B_i} \mathbf{J}_i^c, \tag{5}$$

where $\mathbf{J}^c$ represents the contact Jacobian for a contact point, $\mathcal{B}_c$ denotes the set of bones influencing the corresponding collision mesh, w_{B_i} and $\mathbf{J}_i^c$ are the weight and contact Jacobian for the ith influencing bone at the collision point, respectively. For how the contact Jacobian is used in a general rigid body simulation, please refer to [43].

For a rigid object, the deformation caused by a collision occurs only at the joint locations; hence, its output result is not visually convincing. To improve this, we approximated the collision of a soft-body object by allowing some velocity, which is proportional to the penetrating depth, instead of avoiding the collision immediately [44]. At each frame of the simulation, object deformation becomes apparent when updating the position and orientation of the collision meshes of the object's updated state.

6. Experimental results

Our experiments were performed on an Intel i5-13600KF 3.5 GHz PC with 16 GB of DDR4 memory and an NVIDIA Geforce RTX 1080Ti GPU under the Ubuntu operation system (version 20.04.6 LTS, 64-bit). Our implementation utilized the external software libraries of V-HACD [40] and libccd [42] for the physics-based simulation and OpenGL [45] for rendering. The experimental results are best understood through examples of their use, as described in the subsequent sections and the accompanying video.

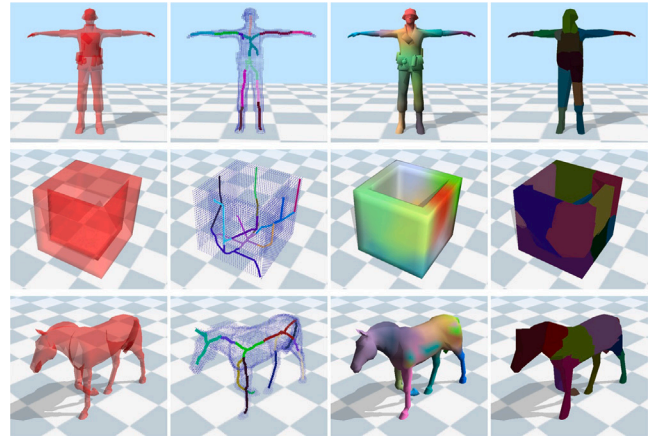


Fig. 6. Input mesh models with rigging results.

Table 2
Rigging performance outcomes.

Generation stage	Generation time (s)		
	Human (25,371[a])	Box (27,870[a])	Horse (33,993[a])
Voxels	0.038	0.041	0.043
Skeleton Graph	0.234	0.536	0.151
Clustering	0.088	0.083	0.111
Voxel Weights	0.015	0.012	0.015
Collision Meshes	1.504	1.194	1.927
Total	1.879	1.866	2.247

[a] Number of voxels in mesh.

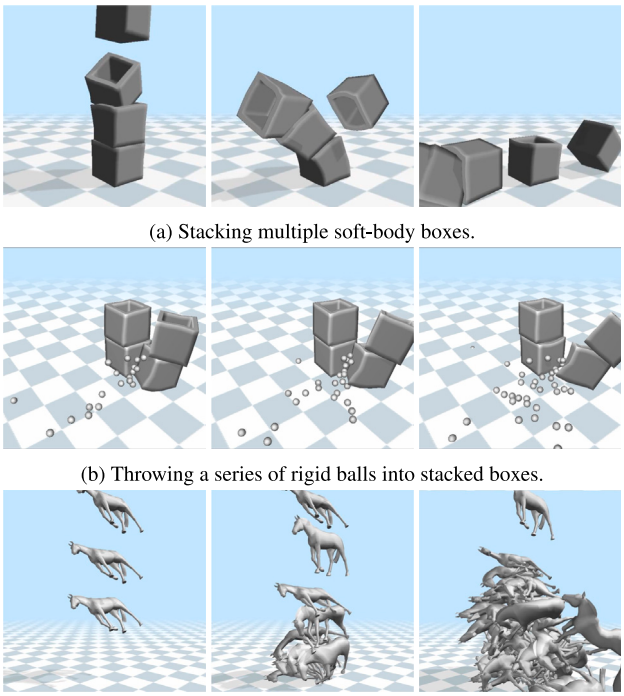
6.1. Rigging performance

Fig. 6 shows the various mesh models used in our experiments. To evaluate the rigging performance, we used the following rigging parameters: $N_c = 12$, $N_k = 4$, $P = 2$, and $\theta_D = 15$. The input meshes need to be watertight to ensure that the voxelization process operates without errors. The information about the input meshes and the resulting skeleton are provided in Table 1.

Table 2 shows the resulting performance for each input model. Overall, it took about 1.866~2.247 s to complete the rigging process, with the generation of collision meshes taking the most time owing to the large size of the graph-structured skeleton. Compared to previous approaches [35,36], our approach does not rely on machine learning, eliminating the need for a training phase, and it successfully performs rigging regardless of the input mesh's shape. Although it can be less efficient in terms of inference speed, the rigging process is completed within seconds, making it practical for its intended applications.

6.2. Physics-based simulation

To demonstrate the capability of our approach, we generated various scenes of simulated interaction between rigid and soft-body objects. For these simulations, we set the time steps for the simulation and rendering to 8.33 ms and 33.3 ms, respectively. For the PD controller, we set K_P and K_D to $300(m/60)$ and $15(m/60)$, respectively, where m is the mass value of the input mesh model. For the added links in Section 4.3, angular springs were used with the internal joints to maintain the angles between adjacent edges for each link. These springs use K_P and K_D values identical to those of the original joints. To prevent the springs from making the object overly rigid, each spring



(a) Stacking multiple soft-body boxes.

(b) Throwing a series of rigid balls into stacked boxes.

(c) Stacking numerous soft-body horses.

Fig. 7. Simulation examples.

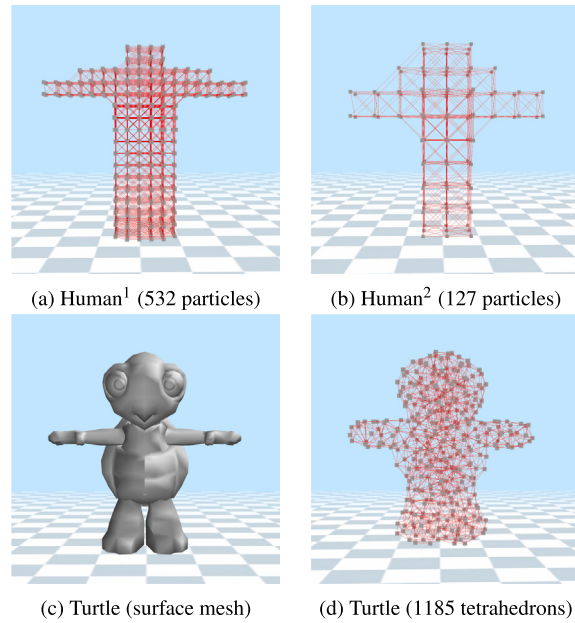
Table 3
Simulation performance: The time required to simulate one second of motion was measured. Our method and XPBD used explicit integration with a simulation rate of 240 Hz, while FEM employed implicit backward Euler integration operating at 100 Hz.

Model	Simulation time (ms)		
	Ours	XPBD	FEM
Human[1]	6.6	1173	N/A
Human[2]		246	
Turtle	10.5	549	598

was modeled with a distance threshold value (i.e., an empirical value of 0.2 m) that allows for a small positional error.

First, a series of soft-body boxes were stacked vertically on the ground. As shown in Fig. 7a, the opening of each box was deformed by the impact force, after which the box opening returned to its original shape due to elasticity. In this example, the mass for each box object was set to 10. Next, a series of rigid balls were thrown into stacked soft-body boxes. As shown in Fig. 7b, the boxes were deformed, and collapsing to the ground owing to the collision force caused by the multiple balls. In this example, the masses of the ball and each box object were set to 5 and 10, respectively. Fig. 7c demonstrates a rapid simulation with numerous deformable objects. One hundred soft-body horses are dropped onto the ground, accumulating on the ground, while some bounce off due to the elasticity of the soft-body materials. This simulation result demonstrates the convincing interaction between soft-body objects in real time. In this experiment, the mass for each horse object was set to 60.

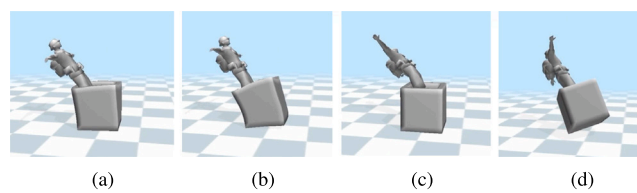
The next experiment compares the computational performance of our method, XPBD, and FEM. For this comparison, we implemented the XPBD algorithm by ourselves and used the open-source library VegaFEM [46,47] for the FEM method. All experiments were conducted on a single thread. As shown in Fig. 8, the character model is approximated as model 1 and 2 as voxel grids with 532 and 127 particles, respectively. Since these are not tetrahedron meshes, FEM simulation was not performed. The turtle model, on the other hand, is a model with 1185 tetrahedrons and 347 vertices. Our method approximated the



(a) Human[1] (532 particles) (b) Human[2] (127 particles)

(c) Turtle (surface mesh) (d) Turtle (1185 tetrahedrons)

Fig. 8. Models for comparison.



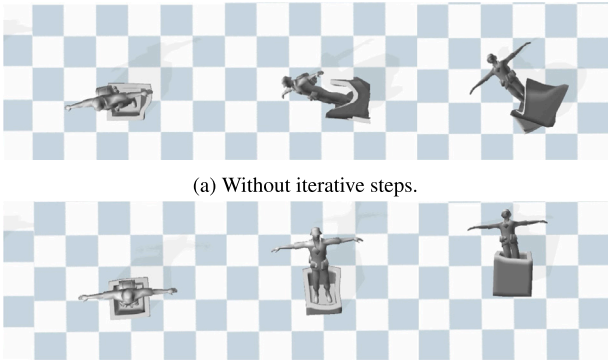
(a) (b) (c) (d)

Fig. 9. Comparison of simulation outcomes with varying material properties: (a) a soft-body human interacting with a stiff soft-body box, (b) a stiff soft-body human interacting with a soft-body box, (c) a soft-body human interacting with a stiff soft-body box, and (d) a stiff body-body human interacting with a stiff soft-body box.

turtle model using 12 bones. Both XPBD and FEM required significantly more elements to represent similar shapes, leading to simulation time differences of at least 18 times. To ensure a fair comparison, parameters were adjusted in all simulators to represent similar material properties. Since we have not yet implemented damping in XPBD, we reduced the damping coefficients in our simulator to approximate the properties of a stiff, bouncy ball (see the accompanying video). In Table 3, the computation time represents the simulation time required to generate one second of results, where values below 1000 indicate real-time simulation capability. For XPBD, incorporating damping representation would further increase this time gap. The VegaFEM does not have an implementation of ground collision responses. Instead, the vertices at the soles of the feet were fixed during the simulation.

Our approach can generate various scenes using the same objects with different material characteristics. To show this, we dropped a human object into a box object, with each object being either rigid or soft-bodied. As examples, we simulated with four different cases: dropping a soft-body human into a soft-body box, dropping a rigid human into a soft-body box, dropping a soft-body human into a rigid box, and dropping a rigid human into a rigid box. Fig. 9 shows the pertinent differences in the deformation changes between the two objects owing to their different material elasticities. In these examples, the mass for both the human and box objects was set to 10.

As an ablation study, we compared the deformation results using the iterative steps introduced in Section 4.1. For this example, a rigid human object was dropped into a soft-body box. As shown in Fig. 10, the box was deformed into an abnormal shape without the iterative



(a) Without iterative steps.

(b) With iterative steps.

Fig. 10. Comparison of simulation of dropping a rigid human into a soft-body box without and with iterative steps. Without iterative steps, the box fails to maintain its shape and unnaturally spreads out.

steps applied. Here, skeleton generation outcomes with and without the iterative steps are compared in Fig. 3. In this example, the mass for both human and box objects was set to 10.

7. Conclusion

In this paper, we introduced an approach by which to realize rapid simulations of soft-body deformation of a 3D mesh model. Using connected rigid objects, our approach maximizes the number of DOFs used, resulting in convincing deformation outcomes, and approximates the simulation process for high-speed performance. Unlike previous approaches that require simulators and preprocessed object models designed by experts, our approach provides a fully automated solution applicable to existing mesh models. This saves time and effort for general users in the current animation production milieu.

There are certain drawbacks to consider when undertaking the deformation of very soft objects using our approach. First, our method focuses on fast simulation of relatively stiff deformable objects. However, it is less suitable for simulating soft objects that requiring large local deformations. We are indirectly representing the material characteristics of soft-body objects by adjusting the spring and damping coefficients between collision meshes. This strategy allows for a large time step by approximating the elasticity of the object surface, but it is less accurate than simulation techniques derived from the energy-based dynamics [25,27]. Our collision handling is simplified by using connected rigid objects, which may not accurately capture complex interactions in highly deformable objects. The biggest drawback of approximating a rigid body as an articulated body is the difficulty in representing volume loss during soft-body collisions. However, since we are targeting relatively stiff soft bodies, we mitigate this issue by allowing slight self-collisions upon impact, creating a visual illusion that minimizes the perceptual drawbacks. In addition, the material parameters of a conventional soft-body simulator cannot be used. This requires an additional tuning process for the parameters in order to determine the appropriate spring and damping coefficients.

Unlike existing machine learning-based approaches, our rigging results cannot predict the actual joint positions of human-like characters. However, rigging for deformable object simulation serves a different purpose and produces different results compared to rigging for character animation. There are no existing studies using rigging to approximate deformable objects, making direct comparisons challenging.

There are several ongoing works that enhance our approach. First, we are improving the accuracy of the simulation by using open source and physics-based simulators such as bullet [48] and PhysicX [49], during which we must automatically convert the material parameters of existing soft-body simulators and apply them to our simulation. Next, we are applying a high-performance parallel system, in this case GPGPU [49], to reduce the simulation time further.

CRediT authorship contribution statement

Moonjun Chung: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Taesoo Kwon:** Writing – review & editing, Validation, Supervision, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Conceptualization. **Yejin Kim:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Methodology, Investigation, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Taesoo Kwon reports financial support was provided by National Research Foundation of Korea. Taesoo Kwon reports financial support was provided by Institute of Information & communications Technology Planning & Evaluation. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the National Research Foundation of Korea (NRF) grant (RS-2023-00222776) and the Institute of Information & communications Technology Planning & Evaluation (IITP), South Korea grant (No. 2021-0-00320), funded by the Korea Government (MSIT).

Appendix A. Supplementary data

Supplementary material related to this article can be found online at https://doi.org/10.1016/j.cag.2025.104202.

Data availability

Data will be made available on request.

References

[1] Llobera J, Charbonnier C. Physics-based character animation for virtual reality. In: Proceedings of 2022 IEEE conference on virtual reality and 3D user interfaces abstracts and workshops. VRW, 2022, p. 56–7. http://dx.doi.org/10.1109/VRW55335.2022.00021.

[2] Macklin M, Müller M, Chentanez N, Kim T-Y. Unified particle physics for real-time applications. ACM Trans Graph 2014;33(4):1–12. http://dx.doi.org/10.1145/2601097.2601152.

[3] Müller M, Macklin M, Chentanez N, Jeschke S, Kim T-Y. Detailed rigid body simulation with extended position based dynamics. Comput Graph Forum 2020;39(8):101–12. http://dx.doi.org/10.1111/cgf.14105.

[4] Abe Y, da Silva M, Popović J. Multiobjective control with frictional contacts. In: Proceedings of the 2007 ACM SIGGRAPH/eurographics symposium on computer animation. 2007, p. 249–58.

[5] da Silva M, Abe Y, Popović J. Interactive simulation of stylized human locomotion. ACM Trans Graph 2008;27(3):1–10. http://dx.doi.org/10.1145/1360612.1360681.

[6] Yin K, Loken K, van de Panne M. SIMBICON: simple biped locomotion control. ACM Trans Graph 2007;26(3):105–es. http://dx.doi.org/10.1145/1276377.1276509.

[7] Coros S, Beaudoin P, van de Panne M. Generalized biped walking control. ACM Trans Graph 2010;29(4):1–9. http://dx.doi.org/10.1145/1778765.1781156.

[8] Kwon T, Hodgins JK. Momentum-mapped inverted pendulum models for controlling dynamic human motions. ACM Trans Graph 2017;36(4):1–14. http://dx.doi.org/10.1145/3072959.2983616.

[9] Peng XB, Berseth G, van de Panne M. Dynamic terrain traversal skills using reinforcement learning. ACM Trans Graph 2015;34(4):1–11. http://dx.doi.org/10.1145/2766910.

[10] Won J, Gopinath D, Hodgins J. A scalable approach to control diverse behaviors for physically simulated characters. ACM Trans Graph 2020;39(4):33:1–33:12. http://dx.doi.org/10.1145/3386569.3392381.

[11] Lee S, Lee J, Lee J. Learning virtual chimeras by dynamic motion reassembly. ACM Trans Graph 2022;41(6):182:1–182:13. http://dx.doi.org/10.1145/3550454.3555489.

[12] Xie Z, Starke S, Ling HY, van de Panne M. Learning soccer juggling skills with layer-wise mixture-of-experts. In: ACM SIGGRAPH 2022 conference proceedings. 2022, p. 1–9. http://dx.doi.org/10.1145/3528233.3530735.

[13] Kim J, Pollard NS. Direct control of simulated nonhuman characters. IEEE Comput Graph Appl 2011;31(4):56–65. http://dx.doi.org/10.1109/MCG.2011.58.

[14] Tan J, Turk G, Liu CK. Soft body locomotion. ACM Trans Graph 2012;31(4):1–11. http://dx.doi.org/10.1145/2185520.2185522.

[15] Kwon T, Kim H, Lee Y. Control of an iguana character using soft-body simulation. IEEE Access 2018;6:77931–9. http://dx.doi.org/10.1109/ACCESS.2018.2884493.

[16] Wang H. GPU-based simulation of cloth wrinkles at submillimeter levels. ACM Trans Graph 2021;40(4):1–14. http://dx.doi.org/10.1145/3450626.3459787.

[17] Wolf T, Cornillère V, Sorkine-Hornung O. Physically-based book simulation with freeform developable surfaces. Comput Graph Forum 2021;40(2):449–60. http://dx.doi.org/10.1111/cgf.142646.

[18] Müller M, Chentanez N. Solid simulation with oriented particles. ACM Trans Graph 2011;30(4):1–10. http://dx.doi.org/10.1145/2010324.1964987.

[19] Romero C, Casas D, Pérez J, Otaduy M. Learning contact corrections for handle-based subspace dynamics. ACM Trans Graph 2021;40(4):1–12. http://dx.doi.org/10.1145/3450626.3459875.

[20] Lan L, Luo R, Fratarcangeli M, Xu W, Wang H, Guo X, Yao J, Yang Y. Medial elastics: Efficient and collision-ready deformation via medial axis transform. ACM Trans Graph 2020;39(3):1–17. http://dx.doi.org/10.1145/3384515.

[21] Kee MH, Um K, Jeong W, Han J. Constrained projective dynamics: Real-time simulation of deformable objects with energy-momentum conservation. ACM Trans Graph 2021;40(4):1–12. http://dx.doi.org/10.1145/3450626.3459878.

[22] Qiao Y-L, Liang J, Koltun V, Lin MC. Differentiable simulation of soft multi-body systems. In: Proceedings of the 35th international conference on neural information processing systems. 2021, p. 17123–35.

[23] Barbič J, James DL. Real-time subspace integration for St. Venant-Kirchhoff deformable models. ACM Trans Graph 2005;24(3):982–90. http://dx.doi.org/10.1145/1073204.1073300.

[24] Gilles B, Bousquet G, Faure F, Pai DK. Frame-based elastic models. ACM Trans Graph 2011;30(2):1–12. http://dx.doi.org/10.1145/1944846.1944855.

[25] Hahn F, Martin S, Thomaszewski B, Sumner R, Coros S, Gross M. Rig-space physics. ACM Trans Graph 2012;31(4):1–8. http://dx.doi.org/10.1145/2185520.2185568.

[26] Xu H, Barbič J. Pose-space subspace dynamics. ACM Trans Graph 2016;35(4):1–14. http://dx.doi.org/10.1145/2897824.2925916.

[27] Zhang JE, Bang S, Levin DIW, Jacobson A. Complementary dynamics. ACM Trans Graph 2020;39(6):1–11. http://dx.doi.org/10.1145/3414685.3417819.

[28] Ju T, Schaefer S, Warren J. Mean value coordinates for closed triangular meshes. ACM Trans Graph 2005;24(3):561–6. http://dx.doi.org/10.1145/1073204.1073229.

[29] Joshi P, Meyer M, DeRose T, Green B, Sanocki T. Harmonic coordinates for character articulation. ACM Trans Graph 2007;26(3):71–es. http://dx.doi.org/10.1145/1276377.1276466.

[30] Lipman Y, Levin D, Cohen-Or D. Green coordinates. ACM Trans Graph 2008;27(3):1–10. http://dx.doi.org/10.1145/1399504.1360677.

[31] Baran I, Popović J. Automatic rigging and animation of 3D characters. ACM Trans Graph 2007;26(3):72–es. http://dx.doi.org/10.1145/1276377.1276467.

[32] Au OK-C, Tai C-L, Chu H-K, Cohen-Or D, Lee T-Y. Skeleton extraction by mesh contraction. ACM Trans Graph 2008;27(3):1–10. http://dx.doi.org/10.1145/1360612.1360643.

[33] Kavan L, Sorkine O. Elasticity-inspired deformers for character articulation. ACM Trans Graph 2012;31(6):196:1–8. http://dx.doi.org/10.1145/2366145.2366215.

[34] Ma J, Wang J, Li J, Zhang D. Real-time skeletonization for sketch-based modeling. Comput Graph 2022;102:56–66. http://dx.doi.org/10.1016/j.cag.2021.11.005.

[35] Liu L, Zheng Y, Tang D, Yuan Y, Fan C, Zhou K. NeuroSkinning: Automatic skin binding for production characters with deep graph networks. ACM Trans Graph 2019;38(4):1–12. http://dx.doi.org/10.1145/3306346.3322969.

[36] Xu Z, Zhou Y, Kalogerakis E, Landreth C, Singh K. RigNet: Neural rigging for articulated characters. ACM Trans Graph 2020;39(4):1–14. http://dx.doi.org/10.1145/3386569.3392379.

[37] Li P, Aberman K, Hanocka R, Liu L, Sorkine-Hornung O, Chen B. Learning skeletal articulations with neural blend shapes. ACM Trans Graph 2021;40(4):1–15. http://dx.doi.org/10.1145/3450626.3459852.

[38] Lee T-C, Kashyap RL, Chu C-N. Building skeleton models via 3-D medial surface axis thinning algorithms. CVGIP, Graph Models Image Process 1994;56(6):462–78. http://dx.doi.org/10.1006/cgip.1994.1042.

[39] Kovar L, Gleicher M. Automated extraction and parameterization of motions in large data sets. ACM Trans Graph 2004;23(3):559–68. http://dx.doi.org/10.1145/1015706.1015760.

[40] V-HACD. 2022, https://github.com/kmammou/v-hacd [Accessed 1 June 2024].

[41] Yin Z, Yin K. Linear time stable PD controllers for physics-based character animation. Comput Graph Forum 2020;39(8):191–200. http://dx.doi.org/10.1111/cgf.14113.

[42] Libccd. 2018, https://github.com/danfis/libccd [Accessed 1 June 2024].

[43] Mirtich B, Canny J. Impulse-based simulation of rigid bodies. In: Proceedings of the 1995 symposium on interactive 3D graphics. 1995, p. 181–ff. http://dx.doi.org/10.1145/199404.199436.

[44] Todorov E. Convex and analytically-invertible dynamics with contacts and constraints: Theory and implementation in MuJoCo. In: Proceedings of 2014 IEEE international conference on robotics and automation. ICRA, 2014, p. 6054–61.

[45] OpenGL. 2017, https://www.opengl.org [Accessed 1 June 2024].

[46] VegaFEM. 2018, https://viterbi-web.usc.edu/~jbarbic/vega [Accessed 11 November 2024].

[47] Teran J, Sifakis E, Irving G, Fedkiw R. Robust quasistatic finite elements and flesh simulation. In: Proceedings of the 2005 ACM SIGGRAPH/eurographics symposium on computer animation. 2005, p. 181–90. http://dx.doi.org/10.1145/1073368.1073394.

[48] Bullet physics SDK. 2022, https://github.com/bulletphysics/bullet3 [Accessed 1 June 2024].

[49] NVIDIA. 2023, https://nvidia.com [Accessed 1 June 2024].